1. **Indexing**

SQL MEDIUM 2 marks

The reports in this lab repeatedly filter and group the exam table by course_id. Create a B-tree index named idx_exam_course_id on that column.

YOUR ANSWER

```
CREATE INDEX idx_exam_course_id
ON exam USING BTREE (course_id);
```

2. **Window Function**

SQL MEDIUM 5 marks

Rank students by marks within each course, highest first. Students with identical marks in the same course must receive the same rank, and the next rank must be skipped. Show course_id, student_id, marks and course_rank. Sort by course_id, then course_rank, then student_id.

YOUR ANSWER

```
SELECT
    course_id,
    student_id,
    marks,
    RANK() OVER (
        PARTITION BY course_id
        ORDER BY marks DESC
    ) AS course_rank
FROM exam
ORDER BY course_id, course_rank, student_id;
```

3. CTE Report

SQL MEDIUM 5 marks

Using a WITH clause containing at least three CTEs, build one row per course that has exam records, showing:

total_students - how many students appeared
above_avg_count - how many scored strictly above that course's own average; show 0 if none did
pass_percentage - the percentage scoring 40 or above, rounded to two decimal places

Sort by course_id.

YOUR ANSWER

```
WITH course_avg AS (
    SELECT
        course_id,
        AVG(marks) AS avg_marks
    FROM exam
    GROUP BY course_id
),
student_counts AS (
    SELECT
        course_id,
        COUNT(*) AS total_students,
        SUM(CASE
            WHEN marks > avg_marks THEN 1
            ELSE 0
        END) AS above_avg_count
    FROM exam
```

1. **Grouping and Aggregation**

SQL MEDIUM 1 mark

For every course that has at least one exam record, show the course_id, how many students appeared, the average marks rounded to two decimal places, and the highest marks. Name the three computed columns students_appeared, average_marks and top_marks. Sort by course_id ascending.

A course with no exam records must not appear in your result.

YOUR ANSWER

```
SELECT
    course_id,
    COUNT(*) AS students_appeared,
    ROUND(AVG(marks), 2) AS average_marks,
    MAX(marks) AS top_marks
FROM exam
GROUP BY course_id
ORDER BY course_id ASC;
```

4. **Self Join**

SQL MEDIUM 2 marks

List every pair of faculty members who work in the same department. Each pair must appear exactly once - do not list a faculty member with themselves, and do not list the same pair twice in reversed order. Name the columns faculty1, faculty2 and department. Sort by department, then by faculty1.

YOUR ANSWER

```
SELECT
    f1.name AS faculty1,
    f2.name AS faculty2,
    f1.department AS department
FROM faculty f1
INNER JOIN faculty f2
    ON f1.department = f2.department
    AND f1.faculty_id < f2.faculty_id
ORDER BY department, faculty1;
```

3. Left Join

SQL MEDIUM 1 mark

List every student together with the number of exams they have appeared for, as exams_taken. A student who has not appeared for any exam must still appear in the result, with a count of 0. Sort by student_id.

YOUR ANSWER

```
SELECT
    s.student_id,
    s.name,
    COUNT(e.exam_id) AS exams_taken
FROM student s
LEFT JOIN exam e
    ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY s.student_id;
```

2. **Inner Join**

SQL MEDIUM 1 mark

For every exam record, show the student's name, the course_name and the marks.
Sort by marks descending, and where marks are equal, by student name ascending.

YOUR ANSWER

```
SELECT
    s.name,
    c.course_name,
    e.marks
FROM exam e
INNER JOIN student s
    ON e.student_id = s.student_id
INNER JOIN course c
    ON e.course_id = c.course_id
ORDER BY
    e.marks DESC,
    s.name ASC;
```

5. **Subquery**

SQL MEDIUM 2 marks

Using a subquery - not a JOIN - list the student_id and name of every student who has no exam record at all. Sort by student_id.

YOUR ANSWER

```
SELECT
    student_id,
    name
FROM student
WHERE student_id NOT IN (
    SELECT student_id
    FROM exam
)
ORDER BY student_id;
```

6. **Set Operation**

SQL MEDIUM 2 marks

Using INTERSECT, list the student_id of every student who both scored 85 or above in at least one exam and appeared for course 201. Sort ascending.

YOUR ANSWER

```
SELECT student_id
FROM exam
WHERE marks >= 85

INTERSECT

SELECT student_id
FROM exam
WHERE course_id = 201

ORDER BY student_id ASC;
```

7. **Views**

SQL MEDIUM 2 marks

Create a view named course_report that shows, for every course with at least one exam record, the course_id, the course_name, and the average marks rounded to two decimal places as average_marks.

YOUR ANSWER

```
CREATE VIEW course_report AS
SELECT
    c.course_id,
    c.course_name,
    ROUND(AVG(e.marks), 2) AS average_marks
FROM course c
JOIN exam e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name
HAVING COUNT(e.exam_id) >= 1;
```

2. Filtering and Sorting

SQL MEDIUM 1 mark

List the student_id, name and branch of every student in the CSE branch, sorted alphabetically by name.

YOUR ANSWER

```
SELECT student_id, name, branch
FROM student
WHERE branch = 'CSE'
ORDER BY name ASC;
```

3. Grouping and Aggregation

SQL MEDIUM 1 mark

For every course that has at least one exam record, show the course_id, how many students appeared, the average marks rounded to two decimal places, and the highest marks. Name the three computed columns students_appeared, average_marks and top_marks. Sort by course_id ascending.

A course with no exam records must not appear in your result.

YOUR ANSWER

```
SELECT
    course_id,
    COUNT(*) AS students_appeared,
    ROUND(AVG(marks), 2) AS average_marks,
    MAX(marks) AS top_marks
FROM exam
GROUP BY course_id
ORDER BY course_id ASC;
```

1. **College Record Management — SQL Operations**

SQL MEDIUM 1 mark

The examination cell has sent three corrections. Write the SQL statements to apply all three, in this order.

1. Rohan (student_id 108) and Aman (student_id 106) both appeared for the AI course (course_id 204), but their records were never entered. Insert these two exam rows:

   exam_id = 312, student_id = 108, course_id = 204, marks = 72
   exam_id = 313, student_id = 106, course_id = 204, marks = 68

2. Kabir (student_id 103) was re-evaluated in Networks (course_id 203). Update his marks in that course to 75.

3. Meera's Operating Systems record (exam_id 307) was entered by mistake. Delete it.

YOUR ANSWER

```
-- 1. Insert exam records for Rohan and Aman
INSERT INTO exam (exam_id, student_id, course_id, marks)
VALUES
    (312, 108, 204, 72),
    (313, 106, 204, 68);

-- 2. Update Kabir's marks in Networks to 75
UPDATE exam
SET marks = 75
WHERE student_id = 103
  AND course_id = 203;

-- 3. Delete Meera's incorrect Operating Systems record
DELETE FROM exam
WHERE exam_id = 307;
```